2024 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C) | 979-8-3503-8976-0/24/$31.00 ©2024 IEEE | DOI: 10.1109/ACSOS-C63493.2024.00057

Automatic Adaptation Rule Optimization via Large Language Models

Yusei Ishimizu[1], Jialong Li[2*], Jinglue Xu[3], Jinyu Cai[2], Hitoshi Iba[3], Kenji Tei[1]
[1]*Tokyo Institute of Technology*, [2]Waseda University, [3]University of Tokyo
[*]Corresponding Author: lijialong@fuji.waseda.jp

***Abstract*—Rule-based adaptation is a foundational approach to self-adaptation, characterized by its human readability and rapid response. However, building high-performance and robust adaptation rules is often a challenge because it essentially involves searching for the optimal design in a complex (variable) space. In response, this paper attempts to employ large language models (LLMs) as an optimizer to construct and optimize adaptation rules, leveraging the common sense and reasoning capabilities inherent in LLMs. Preliminary experiments conducted in SWIM have validated the effectiveness and limitations of our method.**

***Index Terms*—Adaptation Rule, Large Language Models, Rule-based Adaptation, Automatic Design and Optimization**

I. Introduction

Self-adaptive systems are engineered to achieve system goals under dynamic conditions and changing environments. Rule-based adaptation, which is a fundamental approach where the adaptation logic is predefined through rules, offers two significant advantages. Firstly, the human-readable rules enhance the explainability of adaptation. Secondly, since reasoning and planning are not required at runtime, rule-based adaptation is suitable for scenarios that require quick responses [1].

However, designing high-performance and robust adaptation rules is often challenging. Essentially, it is an optimization problem within a complex design space. The development of adaptation rules typically involves two crucial steps. The initial step is the selection of variables, which includes both observed input variables and control variables, essentially designing the dimensions of the search space. The subsequent step focuses on the searching and optimization of rules, essentially finding the optimal solution within the given search space.

Recently, large language models (LLMs) have emerged as significant tools across various research fields [2]. [3] utilizes LLMs for automatic feature engineering, identifying semantically relevant variables from extensive datasets. Meanwhile, [4] explores the potential of LLM-based optimization in the traveling salesman problem. Given the capabilities demonstrated in these studies, we believe LLMs have strong potential for automatically constructing and optimizing adaptive rules.

In this paper, we preliminarily explore the application of LLMs to develop and optimize adaptation rules. Drawing from the established MAPE-K reference architecture, we design a series of prompts tailored for the continuous iteration and optimization of these rules. Additionally, we use SWIM (Simulator for Web Infrastructure and Management) [5] to validate the effectiveness of our method.

II. Proposal

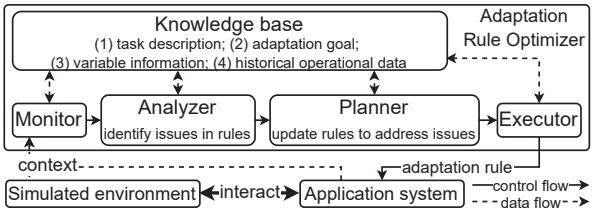

Fig. 1: System Overview.

Overview. We consider the optimization of adaptation rules as processes of automatic computing[1], and use the MAPE-K loop to outline our method, as shown in Fig. 1. In this process, the monitor captures system and environment contexts from the interactions between the application system and its (simulated) environment, and incorporates them into the knowledge base. Following this, the analyzer and the planner generate the updated adaptation rule, and the executor applies the new rules to the application system. In the subsequent part of this section, we will focus primarily on the knowledge, analyzer, and planner components, as our method predominantly involves these three elements.

Knowledge base. When utilizing LLMs, it is essential to provide comprehensive context to support their reasoning. In the knowledge base, we prepare four types of information crucial for rule construction. First, we include an introduction to the application domain, e.g., the operational logic of the application. Second, we define the adaptation goals, clarifying the objectives that the system aims to achieve. Third, we also describe the variables within the application, detailing both observable variables and those the system can control, i.e., defining the system's observable and controllable spaces. Note that in rule construction, LLMs may only use some of these variables or even define new variables using existing ones. Lastly, we incorporate historical operational data, which could serve as feedback to enable LLMs to refine and iterate on adaptation rules.

LLM-driven analyzer and planner. The analyzer's role is to identify potential issues within existing adaptation rules.

[1]This paper only considers design-time rule construction; run-time rule evolution could be represented as a similar process.

Authorized licensed use limited to: Waseda University. Downloaded on March 12,2026 at 12:22:52 UTC from IEEE Xplore. Restrictions apply.

For example, it might analyze the specific reasons for poor performance during certain timesteps and suggest which part of the rules to modify. Utilizing the insights generated by the analyzer, the planner then determines the necessary updates to the adaptation rules. This could include introducing a new observable variable or suggesting adjustments to the values of control variables. It is important to note that both the analyzer and planner rely on the four types of information stored in the knowledge base, especially the historical operational data, to make well-informed decisions.

III. Preliminary Evaluation

A. Experiment Setting

The evaluation addresses the following research questions: What is the performance level of the adaptation rules designed and optimized by LLMs? How does the performance trend change with iterations of the adaptation rules?

Target scenario. We utilize SWIM as the platform to simulate multi-tier web applications, where a load balancer supports multiple servers. Key variables include request arrival rate, server number (adjustable on demand), and the optional content dimmer. The adaptation goal is to maximize a utility function primarily composed of revenue utility minus cost utility. We utilize Clarknet-105m-I70 scenario, which is a more challenging setting due to significant fluctuations in request arrivals per unit time. We omit detailed explanations due to space constraints; interested readers can refer to [5].

LLM and prompt settings. For the experiment, we employ advanced GPT-4 and excellent performance-cost DeepSeek-Coder-V2 (hereafter DS-Coder). Due to the uncertainty in LLMs' outputs, we conduct ten experiments for each LLM, and each experiment includes ten iterations. Furthermore, the adaptation rules are asked to output in C++ language code format, allowing for direct execution within SWIM.

B. Experiment Results and Discussion

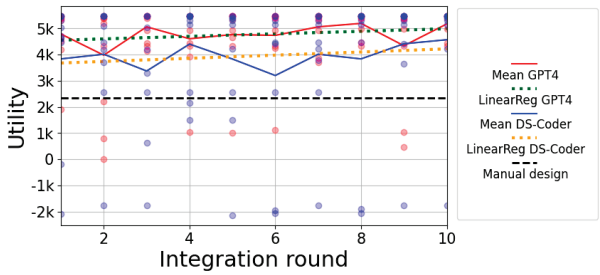

Fig. 2: Results from ten experiments.

Experiment Results. Fig. 2 displays the results of the experiment, revealing several key points. First, both GPT-4 and DS-Coder show superior performance (5.5k in the best cases) compared to the default manual-designed rules within SWIM. The default rules increase servers and adjust the dimmer when the average response time exceeds a threshold (vice versa), and use a total of 8 different variables in the conditionals. Second, due to the strong knowledge and reasoning capabilities of LLMs, the rules generated by LLMs demonstrated high performance even in the first round. In contrast, traditional search or optimization algorithms typically require multiple iterations to achieve acceptable performance. Third, the results of linear regression indicate that the quality of adaptation rules improves with iterations, illustrating the feasibility of LLM-based optimization. However, performance does not necessarily improve with each iteration but shows significant fluctuations (even negative utilities), highlighting the trial-and-error process during iterations.

Discussion and Limitations. We believe our experimental results preliminarily demonstrate the effectiveness of using LLMs as optimizers for designing adaptation rules. However, the results also reveal the limitations of current approaches, namely the inefficiency of using LLMs directly as optimizers, as each iteration only explores one node in the design space. Even with LLMs possessing strong prior knowledge to guide the search, the search remains difficult in a multi-dimensional design space, as the slow response and high invocation cost of LLMs limit the number of iterations. This is also the reason why we do not choose existing optimization algorithms, such as evolutionary computation, as a baseline for comparison, because although existing algorithms often require more iterations for acceptable performance, their per-iteration cost is much lower. Therefore, a significant future research direction is to combine LLM's prior knowledge (as meta-heuristics) with existing search methods to achieve more efficient and economical optimization [6].

IV. Conclusion and Future Work

In this paper, we employ LLMs to automatically design and optimize adaptation rules. Preliminary experiments conducted in the SWIM environment have shown the effectiveness of our method. Future research will primarily focus on two areas. First, we aim to integrate existing optimization algorithms with LLMs to enhance efficiency. Second, we plan to explore more advanced prompt engineering techniques and incorporate fine-tuning to improve the consistency and stability of their outputs. Third, we intend to extend our method to the runtime phase, enabling the automatic evolution of adaptation rules under unforeseen conditions.

References

[1] A. Pandey, G. A. Moreno, J. Cámara, and D. Garlan, "Hybrid planning for decision making in self-adaptive systems," in *SASO*, 2016, pp. 130–139.

[2] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, and K. Tei, "Exploring the potential of large language models in self-adaptive systems," in *SEAMS*, 2024, p. 77–83.

[3] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *NeurIPS*, 2023.

[4] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen, "Large language models as optimizers," in *ICLR*, 2024.

[5] G. A. Moreno, B. Schmerl, and D. Garlan, "Swim: An exemplar for evaluation and comparison of self-adaptation approaches for web applications," in *SEAMS*, 2018, p. 137–143.

[6] J. Cai, J. Xu, J. Li, T. Yamauchi, H. Iba, and K. Tei, "Exploring the improvement of evolutionary computation via large language models," in *GECCO*, 2024.

Authorized licensed use limited to: Waseda University. Downloaded on March 12,2026 at 12:22:52 UTC from IEEE Xplore. Restrictions apply.